

SQL LEVEL 2 — PATTERN QUESTION MASTER SHEET

W3Schools / Northwind-style database • Advanced coding challenge preparation

Core tables: Customers, Orders, OrderDetails, Products, Categories, Suppliers, Employees, Shippers

1. Basic Filtering

Q1. Products costing more than 50

```
SELECT ProductName, Price
FROM Products
WHERE Price > 50;
```

Q2. Products between 20 and 50

```
SELECT ProductName, Price
FROM Products
WHERE Price BETWEEN 20 AND 50;
```

Q3. Names starting with A

```
SELECT ProductName
FROM Products
WHERE ProductName LIKE 'A%';
```

Q4. Names containing 'ch'

```
SELECT ProductName
FROM Products
WHERE ProductName LIKE '%ch%';
```

Q5. Products with NULL supplier

```
SELECT *
FROM Products
WHERE SupplierID IS NULL;
```

2. DISTINCT

Q6. Countries customers are from

```
SELECT DISTINCT Country
FROM Customers;
```

Q7. Number of different countries

```
SELECT COUNT(DISTINCT Country) AS CountryCount
FROM Customers;
```

3. ORDER BY + LIMIT

Q8. Most expensive product

```
SELECT *
FROM Products
ORDER BY Price DESC
LIMIT 1;
```

Q9. Cheapest product

```
SELECT *
FROM Products
ORDER BY Price
LIMIT 1;
```

Q10. Top 5 expensive products

```
SELECT *
FROM Products
ORDER BY Price DESC
LIMIT 5;
```

4. Aggregate Functions

Q11. Average product price

```
SELECT AVG(Price) AS AveragePrice
FROM Products;
```

Q12. Total product value

```
SELECT SUM(Price) AS TotalPrice
FROM Products;
```

Q13. Number of products

```
SELECT COUNT(*) AS ProductCount
FROM Products;
```

5. GROUP BY

Q14. Number of products per category

```
SELECT CategoryID, COUNT(*) AS ProductCount
FROM Products
GROUP BY CategoryID;
```

Q15. Average price per category

```
SELECT CategoryID, AVG(Price) AS AveragePrice
FROM Products
GROUP BY CategoryID;
```

Q16. Maximum price per category

```
SELECT CategoryID, MAX(Price) AS MaxPrice
FROM Products
GROUP BY CategoryID;
```

6. HAVING

Q17. Categories with more than 5 products

```
SELECT CategoryID, COUNT(*) AS ProductCount
FROM Products
GROUP BY CategoryID
HAVING COUNT(*) > 5;
```

Q18. Suppliers with more than 3 products

```
SELECT SupplierID, COUNT(*) AS ProductCount
FROM Products
GROUP BY SupplierID
HAVING COUNT(*) > 3;
```

Q19. Customers with more than 5 orders

```
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerID
HAVING COUNT(*) > 5;
```

7. INNER JOIN

Q20. Product + Category

```
SELECT p.ProductName, c.CategoryName
FROM Products p
INNER JOIN Categories c ON p.CategoryID = c.CategoryID;
```

Q21. Product + Supplier

```
SELECT p.ProductName, s.SupplierName
FROM Products p
INNER JOIN Suppliers s ON p.SupplierID = s.SupplierID;
```

Q22. Customer + Orders

```
SELECT c.CustomerName, o.OrderID, o.OrderDate
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

8. LEFT JOIN — Very Important

Q23. Customers who never placed an order

```
SELECT c.CustomerID, c.CustomerName
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

Q24. Products never ordered

```
SELECT p.ProductID, p.ProductName
FROM Products p
LEFT JOIN OrderDetails od ON p.ProductID = od.ProductID
WHERE od.ProductID IS NULL;
```

9. SELF JOIN

Q25. Employee and manager

```
SELECT e.EmployeeID, e.LastName AS Employee, m.LastName AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ReportsTo = m.EmployeeID;
```

Q26. Employees earning more than their manager

```
SELECT e.LastName AS Employee, e.Salary AS EmployeeSalary,
       m.LastName AS Manager, m.Salary AS ManagerSalary
FROM Employees e
JOIN Employees m ON e.ReportsTo = m.EmployeeID
WHERE e.Salary > m.Salary;
```

10. Above / Below Overall Average

Q27. Products above average price

```
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products);
```

Q28. Products below average price

```
SELECT ProductName, Price
FROM Products
WHERE Price < (SELECT AVG(Price) FROM Products);
```

11. Second / Third Highest

Q29. Second-highest distinct price

```
SELECT MAX(Price)
FROM Products
WHERE Price < (SELECT MAX(Price) FROM Products);
```

Q30. Products having second-highest price

```
SELECT *
FROM Products
WHERE Price = (
    SELECT MAX(Price)
    FROM Products
    WHERE Price < (SELECT MAX(Price) FROM Products)
);
```

Q31. Third-highest distinct price

```
SELECT MAX(Price)
FROM Products
WHERE Price < (
    SELECT MAX(Price)
    FROM Products
    WHERE Price < (SELECT MAX(Price) FROM Products)
);
```

12. Correlated Subquery

Q32. Products more expensive than their category average

```
SELECT p.ProductName, p.CategoryID, p.Price
FROM Products p
WHERE p.Price > (
```

```

SELECT AVG(p2.Price)
FROM Products p2
WHERE p2.CategoryID = p.CategoryID
);

```

Q33. Most expensive product in each category

```

SELECT c.CategoryName, p.ProductName, p.Price
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE p.Price = (
    SELECT MAX(p2.Price)
    FROM Products p2
    WHERE p2.CategoryID = p.CategoryID
);

```

Q34. Cheapest product in each category

```

SELECT c.CategoryName, p.ProductName, p.Price
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE p.Price = (
    SELECT MIN(p2.Price)
    FROM Products p2
    WHERE p2.CategoryID = p.CategoryID
);

```

13. EXISTS / NOT EXISTS

Q35. Customers with at least one order

```

SELECT c.CustomerID, c.CustomerName
FROM Customers c
WHERE EXISTS (
    SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID
);

```

Q36. Customers who never ordered

```

SELECT c.CustomerID, c.CustomerName
FROM Customers c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID
);

```

14. COUNT + JOIN

Q37. Orders per customer

```

SELECT c.CustomerID, c.CustomerName, COUNT(o.OrderID) AS OrderCount
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.CustomerName;

```

Q38. Products per supplier

```

SELECT s.SupplierID, s.SupplierName, COUNT(p.ProductID) AS ProductCount
FROM Suppliers s
JOIN Products p ON s.SupplierID = p.SupplierID
GROUP BY s.SupplierID, s.SupplierName;

```

15. Revenue / Sales Calculations

Q39. Revenue per order detail

```

SELECT od.OrderID, od.ProductID, od.Quantity, p.Price,
    od.Quantity * p.Price AS Revenue
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID;

```

Q40. Total revenue per product

```

SELECT p.ProductID, p.ProductName,
    SUM(od.Quantity * p.Price) AS TotalRevenue
FROM Products p
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID, p.ProductName;

```

Q41. Total revenue per category

```

SELECT c.CategoryName,
       SUM(od.Quantity * p.Price) AS TotalRevenue
FROM Categories c
JOIN Products p ON c.CategoryID = p.CategoryID
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY c.CategoryID, c.CategoryName
ORDER BY TotalRevenue DESC;

```

Q42. Total spending per customer

```

SELECT c.CustomerID, c.CustomerName,
       SUM(od.Quantity * p.Price) AS TotalSpent
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY c.CustomerID, c.CustomerName;

```

Q43. Highest-spending customer

```

SELECT c.CustomerID, c.CustomerName,
       SUM(od.Quantity * p.Price) AS TotalSpent
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY c.CustomerID, c.CustomerName
ORDER BY TotalSpent DESC
LIMIT 1;

```

16. CASE

Q44. Product price classification

```

SELECT ProductName, Price,
       CASE
           WHEN Price >= 100 THEN 'Expensive'
           WHEN Price >= 50 THEN 'Medium'
           ELSE 'Cheap'
       END AS PriceCategory
FROM Products;

```

Q45. Count expensive products

```

SELECT SUM(CASE WHEN Price >= 100 THEN 1 ELSE 0 END) AS ExpensiveProducts
FROM Products;

```

Q46. Count products by price category

```

SELECT CASE
           WHEN Price >= 100 THEN 'Expensive'
           WHEN Price >= 50 THEN 'Medium'
           ELSE 'Cheap'
       END AS PriceCategory,
       COUNT(*) AS ProductCount
FROM Products
GROUP BY CASE
           WHEN Price >= 100 THEN 'Expensive'
           WHEN Price >= 50 THEN 'Medium'
           ELSE 'Cheap'
       END;

```

17. Window Functions — ROW_NUMBER / RANK / DENSE_RANK

Q47. ROW_NUMBER by price

```

SELECT ProductName, Price,
       ROW_NUMBER() OVER (ORDER BY Price DESC) AS RowNum
FROM Products;

```

Q48. RANK by price

```

SELECT ProductName, Price,
       RANK() OVER (ORDER BY Price DESC) AS PriceRank
FROM Products;

```

Q49. DENSE_RANK by price

```

SELECT ProductName, Price,
       DENSE_RANK() OVER (ORDER BY Price DESC) AS PriceRank
FROM Products;

```

Q50. Top 3 expensive products per category

```
SELECT *
FROM (
    SELECT p.ProductName, p.CategoryID, p.Price,
           ROW_NUMBER() OVER (
               PARTITION BY p.CategoryID ORDER BY p.Price DESC
           ) AS rn
    FROM Products p
) x
WHERE rn <= 3;
```

Q51. Highest product(s) per category with ties

```
SELECT *
FROM (
    SELECT p.ProductName, p.CategoryID, p.Price,
           RANK() OVER (
               PARTITION BY p.CategoryID ORDER BY p.Price DESC
           ) AS rnk
    FROM Products p
) x
WHERE rnk = 1;
```

Q52. Second-highest distinct price per category

```
SELECT *
FROM (
    SELECT p.ProductName, p.CategoryID, p.Price,
           DENSE_RANK() OVER (
               PARTITION BY p.CategoryID ORDER BY p.Price DESC
           ) AS rnk
    FROM Products p
) x
WHERE rnk = 2;
```

18. Window Aggregates

Q53. Category average alongside each product

```
SELECT ProductName, CategoryID, Price,
       AVG(Price) OVER (PARTITION BY CategoryID) AS CategoryAverage
FROM Products;
```

Q54. Products above category average

```
SELECT *
FROM (
    SELECT ProductName, CategoryID, Price,
           AVG(Price) OVER (PARTITION BY CategoryID) AS CategoryAverage
    FROM Products
) x
WHERE Price > CategoryAverage;
```

Q55. Maximum price per category using window

```
SELECT *
FROM (
    SELECT ProductName, CategoryID, Price,
           MAX(Price) OVER (PARTITION BY CategoryID) AS MaxCategoryPrice
    FROM Products
) x
WHERE Price = MaxCategoryPrice;
```

Q56. Minimum price per category using window

```
SELECT *
FROM (
    SELECT ProductName, CategoryID, Price,
           MIN(Price) OVER (PARTITION BY CategoryID) AS MinCategoryPrice
    FROM Products
) x
WHERE Price = MinCategoryPrice;
```

Q57. Previous product price

```
SELECT ProductName, Price,
       LAG(Price) OVER (ORDER BY Price) AS PreviousPrice
FROM Products;
```

Q58. Next product price

```

SELECT ProductName, Price,
       LEAD(Price) OVER (ORDER BY Price) AS NextPrice
FROM Products;

```

Q59. Price difference from previous row

```

SELECT ProductName, Price,
       Price - LAG(Price) OVER (ORDER BY Price) AS PriceDifference
FROM Products;

```

Q60. Running total of quantity

```

SELECT OrderID, Quantity,
       SUM(Quantity) OVER (ORDER BY OrderID) AS RunningTotal
FROM OrderDetails;

```

Q61. Running average of quantity

```

SELECT OrderID, Quantity,
       AVG(Quantity) OVER (ORDER BY OrderID) AS RunningAverage
FROM OrderDetails;

```

Q62. Total quantity per product alongside every row

```

SELECT ProductID, OrderID, Quantity,
       SUM(Quantity) OVER (PARTITION BY ProductID) AS TotalProductQuantity
FROM OrderDetails;

```

Q63. Percentage of category total

```

SELECT ProductName, CategoryID, Price,
       Price * 100.0 / SUM(Price) OVER (PARTITION BY CategoryID) AS PercentageOfCategory
FROM Products;

```

19. CTEs

Q64. CTE for products above average

```

WITH AvgPrice AS (
    SELECT AVG(Price) AS AveragePrice FROM Products
)
SELECT ProductName, Price
FROM Products p
CROSS JOIN AvgPrice a
WHERE p.Price > a.AveragePrice;

```

Q65. Customer spending above 1000

```

WITH CustomerSpending AS (
    SELECT c.CustomerID, c.CustomerName,
           SUM(od.Quantity * p.Price) AS TotalSpent
    FROM Customers c
    JOIN Orders o ON c.CustomerID = o.CustomerID
    JOIN OrderDetails od ON o.OrderID = od.OrderID
    JOIN Products p ON od.ProductID = p.ProductID
    GROUP BY c.CustomerID, c.CustomerName
)
SELECT *
FROM CustomerSpending
WHERE TotalSpent > 1000;

```

Q66. Multiple CTEs + ranking

```

WITH CategoryRevenue AS (
    SELECT c.CategoryID, c.CategoryName,
           SUM(od.Quantity * p.Price) AS Revenue
    FROM Categories c
    JOIN Products p ON c.CategoryID = p.CategoryID
    JOIN OrderDetails od ON p.ProductID = od.ProductID
    GROUP BY c.CategoryID, c.CategoryName
),
RankedCategories AS (
    SELECT *, RANK() OVER (ORDER BY Revenue DESC) AS rnk
    FROM CategoryRevenue
)
SELECT *
FROM RankedCategories
WHERE rnk = 1;

```

Q67. Customer(s) with maximum orders

```

SELECT *
FROM (

```

```

SELECT c.CustomerID, c.CustomerName,
       COUNT(o.OrderID) AS OrderCount,
       RANK() OVER (ORDER BY COUNT(o.OrderID) DESC) AS rnk
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.CustomerName
) x
WHERE rnk = 1;

```

Q68. Top 2 customers per country

```

SELECT *
FROM (
    SELECT c.Country, c.CustomerID, c.CustomerName,
           COUNT(o.OrderID) AS OrderCount,
           ROW_NUMBER() OVER (
               PARTITION BY c.Country
               ORDER BY COUNT(o.OrderID) DESC
           ) AS rn
    FROM Customers c
    JOIN Orders o ON c.CustomerID = o.CustomerID
    GROUP BY c.Country, c.CustomerID, c.CustomerName
) x
WHERE rn <= 2;

```

20. UNION

Q69. Unique countries from customers and suppliers

```

SELECT Country FROM Customers
UNION
SELECT Country FROM Suppliers;

```

Q70. Countries including duplicates

```

SELECT Country FROM Customers
UNION ALL
SELECT Country FROM Suppliers;

```

21. String Functions

Q71. Upper/lower/length

```

SELECT ProductName,
       UPPER(ProductName) AS UpperName,
       LOWER(ProductName) AS LowerName,
       LENGTH(ProductName) AS NameLength
FROM Products;

```

Q72. CONCAT

```

SELECT CONCAT(ProductName, ' - ', Price) AS ProductInfo
FROM Products;

```

Q73. SUBSTRING

```

SELECT ProductName, SUBSTRING(ProductName, 1, 3) AS ShortName
FROM Products;

```

22. Date Patterns

Q74. Orders from 2025

```

SELECT * FROM Orders
WHERE YEAR(OrderDate) = 2025;

```

Q75. Orders after Jan 1, 2025

```

SELECT * FROM Orders
WHERE OrderDate >= '2025-01-01';

```

Q76. Orders grouped by year

```

SELECT YEAR(OrderDate) AS OrderYear,
       COUNT(*) AS OrderCount
FROM Orders
GROUP BY YEAR(OrderDate)
ORDER BY OrderYear;

```

Q77. Orders grouped by month


```

SELECT YEAR(OrderDate) AS OrderYear,
       MONTH(OrderDate) AS OrderMonth,
       COUNT(*) AS OrderCount
FROM Orders
GROUP BY YEAR(OrderDate), MONTH(OrderDate)
ORDER BY OrderYear, OrderMonth;

```

Q78. Last 30 days

```

SELECT * FROM Orders
WHERE OrderDate >= CURRENT_DATE() - INTERVAL 30 DAY;

```

23. NULL / Utility Functions

Q79. COALESCE

```

SELECT CustomerName,
       COALESCE(Country, 'Unknown') AS Country
FROM Customers;

```

Q80. NULLIF for division safety

```

SELECT Quantity / NULLIF(Discount, 0) AS Result
FROM OrderDetails;

```

24. IN / NOT IN / ALL / ANY

Q81. Products supplied by USA suppliers

```

SELECT ProductName
FROM Products
WHERE SupplierID IN (
    SELECT SupplierID FROM Suppliers WHERE Country = 'USA'
);

```

Q82. Products not supplied by USA suppliers

```

SELECT ProductName
FROM Products
WHERE SupplierID NOT IN (
    SELECT SupplierID FROM Suppliers WHERE Country = 'USA'
);

```

Q83. More expensive than every Category 1 product

```

SELECT ProductName, Price
FROM Products
WHERE Price > ALL (
    SELECT Price FROM Products WHERE CategoryID = 1
);

```

Q84. More expensive than at least one Category 1 product

```

SELECT ProductName, Price
FROM Products
WHERE Price > ANY (
    SELECT Price FROM Products WHERE CategoryID = 1
);

```

25. Views / Procedures / Functions / Triggers / Transactions

Q85. Create a view

```

CREATE VIEW ExpensiveProducts AS
SELECT ProductID, ProductName, Price
FROM Products
WHERE Price > 100;

```

Q86. Stored procedure

```

DELIMITER //
CREATE PROCEDURE GetProductsByCategory(IN catID INT)
BEGIN
    SELECT * FROM Products WHERE CategoryID = catID;
END //
DELIMITER ;

CALL GetProductsByCategory(1);

```

Q87. Stored function

```

DELIMITER //
CREATE FUNCTION AnnualAmount(amount DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN amount * 12;
END //
DELIMITER ;

```

Q88. Trigger

```

CREATE TRIGGER before_product_insert
BEFORE INSERT ON Products
FOR EACH ROW
SET NEW.Price = ABS(NEW.Price);

```

Q89. Transaction

```

START TRANSACTION;

UPDATE Products
SET Price = Price + 10
WHERE CategoryID = 1;

COMMIT;

-- or use ROLLBACK;

```

26. Hard Multi-Step Patterns

Q90. Customers whose spending is above average customer spending

```

WITH CustomerSpending AS (
    SELECT c.CustomerID, c.CustomerName,
           SUM(od.Quantity * p.Price) AS TotalSpent
    FROM Customers c
    JOIN Orders o ON c.CustomerID = o.CustomerID
    JOIN OrderDetails od ON o.OrderID = od.OrderID
    JOIN Products p ON od.ProductID = p.ProductID
    GROUP BY c.CustomerID, c.CustomerName
)
SELECT *
FROM CustomerSpending
WHERE TotalSpent > (
    SELECT AVG(TotalSpent) FROM CustomerSpending
);

```

Q91. Product bought by the most customers

```

SELECT *
FROM (
    SELECT p.ProductID, p.ProductName,
           COUNT(DISTINCT o.CustomerID) AS CustomerCount,
           RANK() OVER (
               ORDER BY COUNT(DISTINCT o.CustomerID) DESC
           ) AS rnk
    FROM Products p
    JOIN OrderDetails od ON p.ProductID = od.ProductID
    JOIN Orders o ON od.OrderID = o.OrderID
    GROUP BY p.ProductID, p.ProductName
) x
WHERE rnk = 1;

```

Q92. Top product by revenue per category

```

WITH ProductRevenue AS (
    SELECT p.ProductID, p.ProductName, p.CategoryID,
           SUM(od.Quantity * p.Price) AS Revenue
    FROM Products p
    JOIN OrderDetails od ON p.ProductID = od.ProductID
    GROUP BY p.ProductID, p.ProductName, p.CategoryID
),
RankedProducts AS (
    SELECT *, RANK() OVER (
        PARTITION BY CategoryID ORDER BY Revenue DESC
    ) AS rnk
    FROM ProductRevenue
)
SELECT *
FROM RankedProducts
WHERE rnk = 1;

```

■ THE 15 PATTERNS YOU ABSOLUTELY MUST KNOW

1. Above average

```
WHERE Price > (SELECT AVG(Price) FROM Products)
```

2. Second highest

```
MAX(Price) WHERE Price < (SELECT MAX(Price) FROM Products)
```

3. Correlated subquery

```
WHERE p2.CategoryID = p.CategoryID
```

4. Maximum per group

```
WHERE p.Price = (SELECT MAX(p2.Price) FROM Products p2 WHERE p2.CategoryID = p.CategoryID)
```

5. Missing records

```
LEFT JOIN ... WHERE right_table.id IS NULL
```

6. Group filtering

```
GROUP BY ... HAVING COUNT(*) > N
```

7. Ranking

```
RANK() OVER (ORDER BY value DESC)
```

8. Top N per group

```
ROW_NUMBER() OVER (PARTITION BY category ORDER BY value DESC)
```

9. Ties

```
RANK()
```

10. Distinct ranking

```
DENSE_RANK()
```

11. Group average without GROUP BY

```
AVG(value) OVER (PARTITION BY group)
```

12. Previous row

```
LAG(value) OVER (...)
```

13. Next row

```
LEAD(value) OVER (...)
```

14. Running total

```
SUM(value) OVER (ORDER BY date)
```

15. Multi-step problem

```
CTE → GROUP BY → WINDOW FUNCTION → FILTER
```

■ SQL EXAM TRAPS

NULL

- WHERE x = NULL
- WHERE x IS NULL

Aggregate filtering

- WHERE COUNT(*) > 5
- HAVING COUNT(*) > 5

Derived table alias

- FROM (SELECT ...)
- FROM (SELECT ...) x

WHERE vs HAVING

```
WHERE → filters rows  
GROUP BY → creates groups  
HAVING → filters groups
```

Ranking

ROW_NUMBER → unique numbering
RANK → ties + gaps
DENSE_RANK → ties + no gaps

UNION

UNION → removes duplicates
UNION ALL → keeps duplicates

COUNT

COUNT(*) → counts rows
COUNT(column) → does not count NULL

■ FINAL MENTAL CHECKLIST

1. One table or JOINS?
2. Filtering rows? → WHERE
3. Calculating groups? → GROUP BY
4. Filtering groups? → HAVING
5. Comparing against another result? → Subquery / CTE
6. "Each category/country/department"? → PARTITION BY
7. "Top N per..."? → ROW_NUMBER / RANK
8. "Highest/lowest with ties"? → RANK
9. "Previous/next"? → LAG / LEAD
10. Multiple stages? → CTE